

Documento C — Técnicas e Casos de Teste

Projeto: Finansme Flutter

Tecnologia: Flutter / Dart

Norma aplicada: ISO/IEC/IEEE 29119-4

1. Técnicas Utilizadas

Técnica	Finalidade
Particionamento de Equivalência	Separar entradas válidas e inválidas em classes (cadastros, campos obrigatórios)
Valor Limite	Validar fronteiras de valores numéricos e campos vazios (saldo zero, senha curta)
Transição de Estado	Validar mudança de estado do sistema (saldo após receita/despesa, recursos antes)
Teste Baseado em Cenário	Validar fluxos compostos por múltiplas chamadas (saldo + lançamento + saldo)
Tabela de Decisão	Validar combinações de presença/ausência de token e validade de credenciais
Teste de Contrato	Validar estrutura e tipos dos payloads de requisição e resposta JSON

2. Derivação das Condições de Teste

Módulo User — Cadastro (Signin)

CT01 — Validar cadastro de usuário com dados válidos

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Existe uma classe válida de entrada com nome, e-mail bem formado, senha de tamanho mínimo e status ativo.

TC01 — Cadastro de usuário com dados válidos

Método: POST — Endpoint: /signin

Entradas:

```
name = "Maria Silva"
email = "maria@teste.com"
password_hash = "12345678"
active = true
```

Resultado Esperado:

- HTTP 201 Created
- message = "Usuário cadastrado com sucesso."

CT02 — Validar bloqueio de cadastro com e-mail duplicado

Técnica Utilizada:

- Transição de Estado

Justificativa:

O sistema muda do estado "e-mail livre" para "e-mail já cadastrado"; nova tentativa de cadastro deve ser bloqueada.

TC02 — Cadastro com e-mail já existente

Método: POST — Endpoint: /signin

Pré-condição:

Usuário com e-mail maria@teste.com já cadastrado.

Entradas:

```
name = "Maria Silva"
email = "maria@teste.com"
password_hash = "12345678"
active = true
```

Resultado Esperado:

- HTTP 409 Conflict
- message = "E-mail já cadastrado."

CT03 — Validar rejeição de cadastro com campo obrigatório ausente

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Body sem o campo name pertence à classe inválida (campo obrigatório ausente).

TC03 — Cadastro sem nome no body

Método: POST — Endpoint: /signin

Entradas:

```
email = "maria@teste.com"
password_hash = "12345678"
active = true
```

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "name"

CT04 — Validar rejeição de cadastro com e-mail inválido**Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

Valor sem @ e sem domínio pertence à classe inválida de e-mails.

TC04 — Cadastro com formato de e-mail inválido

Método: POST — Endpoint: /signin

Entradas:

```
name = "Maria Silva"
email = "email-invalido"
password_hash = "12345678"
active = true
```

Resultado Esperado:

- HTTP 400 Bad Request
- message = "Formato de e-mail inválido."

CT05 — Validar rejeição de cadastro com senha muito curta**Técnica Utilizada:**

- Valor Limite

Justificativa:

Senha com 3 caracteres está abaixo do limite inferior aceito (8 caracteres).

TC05 — Cadastro com senha menor que 8 caracteres

Método: POST — Endpoint: /signin

Entradas:

```
name = "Maria Silva"
```

```
email = "maria@teste.com"
password_hash = "123"
active = true
```

Resultado Esperado:

- HTTP 422 Unprocessable Entity
- message = "A senha deve ter no mínimo 8 caracteres."

CT06 — Validar rejeição de cadastro com body vazio**Técnica Utilizada:**

- Valor Limite

Justificativa:

Body vazio é o caso limite inferior de ausência de dados.

TC06 — Cadastro com body vazio

Método: POST – Endpoint: /signin

Entradas:

```
body = {}
```

Resultado Esperado:

- HTTP 400 Bad Request
- message = "Corpo da requisição inválido."

Módulo User — Login**CT07 — Validar login com credenciais válidas****Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

Combinação de e-mail e senha válidos pertence à classe válida; espera-se token no retorno.

TC07 — Login com credenciais corretas

Método: POST – Endpoint: /login

Entradas:

```
email = "maria@teste.com"
password_hash = "12345678"
```

Resultado Esperado:

- HTTP 200 OK
- message = "Login realizado com sucesso."
- token = "token-ct-finansme"

CT08 — Validar rejeição de login com senha incorreta**Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

Senha incorreta pertence à classe inválida; sistema deve negar autenticação.

TC08 — Login com senha incorreta

Método: POST – Endpoint: /login

Entradas:

email = "maria@teste.com"

password_hash = "senhaerrada"

Resultado Esperado:

- HTTP 401 Unauthorized
- message = "E-mail ou senha inválidos."

CT09 — Validar rejeição de login com usuário inexistente**Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

E-mail não cadastrado pertence à classe inválida.

TC09 — Login com e-mail não cadastrado

Método: POST – Endpoint: /login

Entradas:

email = "naoexiste@teste.com"

password_hash = "12345678"

Resultado Esperado:

- HTTP 401 Unauthorized
- message = "E-mail ou senha inválidos."

CT10 — Validar rejeição de login com campos vazios**Técnica Utilizada:**

- Particionamento de Equivalência
- Valor Limite

Justificativa:

Campos vazios representam partição inválida e limite inferior de entradas aceitáveis.

TC10 — Login com e-mail e senha em branco

Método: POST – Endpoint: /login

Entradas:

email = ""

password_hash = ""

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "e-mail"

CT11 — Validar rejeição de login com e-mail ausente no body

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Ausência de campo obrigatório (email) pertence à classe inválida.

TC11 — Login sem o campo email

Método: POST – Endpoint: /login

Entradas:

password_hash = "12345678"

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "email"

CT12 — Validar presença e tipo do token na resposta de login

Técnica Utilizada:

- Teste de Contrato

Justificativa:

O contrato da resposta deve garantir as chaves message e token, ambas com valores não vazios.

TC12 — Verificação das chaves de resposta do login

Método: POST – Endpoint: /login

Entradas:

email = "maria@teste.com"

password_hash = "12345678"

Resultado Esperado:

- HTTP 200 OK
- response contém as chaves message e token
- token é String não vazia

Módulo User — Atualização (/alter)

CT13 — Validar atualização de usuário autenticado

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Body válido com token Bearer válido representa a classe válida de atualização.

TC13 — Atualização de dados com token válido

Método: PUT – Endpoint: /alter

Entradas:

Authorization: Bearer token-ct-finansme
name = "Maria Souza"
password_hash = "87654321"
active = true

Resultado Esperado:

- HTTP 200 OK
- message = "Dados atualizados com sucesso."

CT14 — Validar rejeição de atualização sem token de autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Ausência de token combinada com chamada a recurso protegido deve resultar em 401.

TC14 — Atualização sem header Authorization

Método: PUT – Endpoint: /alter

Entradas:

name = "Maria Souza"
password_hash = "87654321"
active = true

Resultado Esperado:

- HTTP 401 Unauthorized
- message = "Token de autenticação ausente ou inválido."

CT15 — Validar rejeição de atualização com token expirado

Técnica Utilizada:

- Transição de Estado

Justificativa:

Token muda do estado "válido" para "expirado"; sistema deve recusar requisições com token expirado.

TC15 — Atualização com token expirado

Método: PUT – Endpoint: /alter

Entradas:

Authorization: Bearer token-expirado
name = "Maria Souza"
password_hash = "87654321"
active = true

Resultado Esperado:

- HTTP 401 Unauthorized
- message contém "Token"

CT16 — Validar rejeição de atualização com dados inválidos

Técnica Utilizada:

- Particionamento de Equivalência
- Valor Limite

Justificativa:

Campos name e password_hash vazios pertencem à classe inválida (limite inferior).

TC16 — Atualização com campos vazios

Método: PUT — Endpoint: /alter

Entradas:

Authorization: Bearer token-ct-finansme

name = ""

password_hash = ""

Resultado Esperado:

- HTTP 400 Bad Request
- message = "Dados inválidos para atualização."

CT17 — Validar atualização parcial — apenas o nome

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Atualização parcial é variação válida do contrato (apenas o subconjunto enviado deve ser alterado).

TC17 — Atualização enviando somente o campo name

Método: PUT — Endpoint: /alter

Entradas:

Authorization: Bearer token-ct-finansme

name = "Novo Nome"

Resultado Esperado:

- HTTP 200 OK
- message = "Dados atualizados com sucesso."

Módulo Account — Cadastro de conta

CT18 — Validar cadastro de conta com dados válidos

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Descrição preenchida e saldo positivo pertencem à classe válida de entrada.

TC18 — Cadastrar conta "Carteira" com saldo 150,00

Método: POST — Endpoint: /accounts

Entradas:

description = "Carteira"

balance = 150.00

Resultado Esperado:

- HTTP 201 Created
- message = "Conta cadastrada com sucesso."

CT19 — Validar cadastro de conta com saldo zero**Técnica Utilizada:**

- Valor Limite

Justificativa:

Saldo igual a zero é o limite inferior aceito (saldo não negativo).

TC19 — Cadastrar conta "Poupança" com saldo 0,00

Método: POST — Endpoint: /accounts

Entradas:

description = "Poupança"

balance = 0.00

Resultado Esperado:

- HTTP 201 Created
- message = "Conta cadastrada com sucesso."

CT20 — Validar rejeição de cadastro de conta sem descrição**Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

Body sem o campo description pertence à classe inválida.

TC20 — Cadastro de conta sem o campo description

Método: POST — Endpoint: /accounts

Entradas:

balance = 100.00

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "description"

CT21 — Validar rejeição de cadastro de conta com saldo negativo**Técnica Utilizada:**

- Valor Limite

Justificativa:

Saldo -50,00 está abaixo do limite inferior permitido (zero).

TC21 — Cadastro de conta com saldo negativo

Método: POST — Endpoint: /accounts

Entradas:

description = "Conta inválida"

balance = -50.00

Resultado Esperado:

- HTTP 400 Bad Request
- message = "O saldo inicial não pode ser negativo."

CT22 — Validar rejeição de cadastro de conta sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC22 — Cadastro de conta sem header Authorization

Método: POST — Endpoint: /accounts

Entradas:

description = "Conta sem auth"

balance = 100.00

Resultado Esperado:

- HTTP 401 Unauthorized
- message = "Token de autenticação ausente ou inválido."

CT23 — Validar cadastro de conta com autenticação válida

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Token Bearer válido + dados válidos representam a classe válida do recurso protegido.

TC23 — Cadastro de conta com Bearer Token correto

Método: POST — Endpoint: /accounts

Entradas:

Authorization: Bearer token-ct-finansme

description = "Conta Corrente"

balance = 500.00

Resultado Esperado:

- HTTP 201 Created
- message = "Conta cadastrada com sucesso."

Módulo Account — Listagem de contas

CT24 — Validar listagem de contas do usuário autenticado

Técnica Utilizada:

- Particionamento de Equivalência
- Teste de Contrato

Justificativa:

Usuário autenticado pertence à classe válida e deve receber suas contas com o contrato esperado.

TC24 — Listar contas com token válido

Método: GET — Endpoint: /user-accounts

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- message é String
- data é lista não vazia
- cada item contém id_account, description e balance

CT25 — Validar listagem de múltiplas contas do usuário

Técnica Utilizada:

- Teste Baseado em Cenário

Justificativa:

Cenário em que o usuário possui mais de uma conta cadastrada; a lista deve refletir todas.

TC25 — Listar 3 contas do usuário

Método: GET — Endpoint: /user-accounts

Pré-condição:

Usuário possui 3 contas cadastradas.

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data.length = 3

CT26 — Validar lista vazia quando o usuário não possui contas

Técnica Utilizada:

- Valor Limite

Justificativa:

Cenário limite: nenhuma conta cadastrada deve retornar lista vazia (não erro).

TC26 — Listar contas de usuário sem contas

Método: GET — Endpoint: /user-accounts

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data = []
- message = "Nenhuma conta encontrada."

CT27 — Validar rejeição de listagem de contas sem autenticação**Técnica Utilizada:**

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC27 — Listar contas sem header Authorization

Método: GET — Endpoint: /user-accounts

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized
- message = "Token de autenticação ausente ou inválido."

CT28 — Validar estrutura completa de cada conta retornada**Técnica Utilizada:**

- Teste de Contrato

Justificativa:

Cada conta deve conter os campos id_account (int), description (String) e balance (num).

TC28 — Verificar tipos dos campos da conta listada

Método: GET — Endpoint: /user-accounts

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- id_account é int
- description é String
- balance é num

Módulo Account — Saldo do usuário**CT29 — Validar consulta de saldo do usuário autenticado****Técnica Utilizada:**

- Particionamento de Equivalência

- Teste de Contrato

Justificativa:

Usuário autenticado pertence à classe válida; response deve conter message (String) e balance (num).

TC29 — Consultar saldo com token válido

Método: GET — Endpoint: /user-balance

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- message é String
- balance é num

CT30 — Validar aumento de saldo após lançamento de receita

Técnica Utilizada:

- Transição de Estado
- Teste Baseado em Cenário

Justificativa:

Estado do saldo deve transitar para um valor maior após o registro de uma receita (type_movement_id = 1).

TC30 — GET saldo → POST receita → GET saldo

Método: GET/POST — Endpoint: /user-balance e /financial-movements

Entradas:

Saldo inicial = 150,00

POST /financial-movements com type_movement_id=1, value=100,00, reason="Salário"

Resultado Esperado:

- saldoAposReceita > saldoInicial

CT31 — Validar redução de saldo após lançamento de despesa

Técnica Utilizada:

- Transição de Estado
- Teste Baseado em Cenário

Justificativa:

Estado do saldo deve transitar para um valor menor após o registro de uma despesa (type_movement_id = 2).

TC31 — GET saldo → POST despesa → GET saldo

Método: GET/POST — Endpoint: /user-balance e /financial-movements

Entradas:

Saldo inicial = 150,00

POST /financial-movements com type_movement_id=2, value=40,00, reason="Mercado"

Resultado Esperado:

- saldoAposDespesa < saldoInicial

CT32 — Validar impacto sequencial de receita e despesa

Técnica Utilizada:

- Teste Baseado em Cenário
- Transição de Estado

Justificativa:

Cenário composto que valida múltiplas transições consecutivas do saldo.

TC32 — Sequência saldo → receita → saldo → despesa → saldo

Método: GET/POST – Endpoint: /user-balance e /financial-movements

Entradas:

Saldo inicial = 150,00

POST receita value=100,00

POST despesa value=40,00

Resultado Esperado:

- saldoAposReceita > saldoInicial
- saldoAposDespesa < saldoAposReceita

CT33 — Validar rejeição de consulta de saldo sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC33 — Consulta de saldo sem Authorization

Método: GET – Endpoint: /user-balance

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized
- message = "Token de autenticação ausente ou inválido."

Módulo Account Plan — Planos de conta

CT34 — Validar cadastro de plano de conta com dados válidos

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Descrição preenchida + token válido pertencem à classe válida.

TC34 — Cadastrar plano "Alimentação"

Método: POST – Endpoint: /account-plans

Entradas:

Authorization: Bearer token-ct-finansme
description = "Alimentação"

Resultado Esperado:

- HTTP 201 Created
- message = "Plano de conta cadastrado com sucesso."

CT35 — Validar rejeição de plano de conta sem descrição

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Body sem o campo description pertence à classe inválida.

TC35 — Cadastro de plano com body {}

Método: POST – Endpoint: /account-plans

Entradas:

Authorization: Bearer token-ct-finansme
body = {}

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "description"

CT36 — Validar rejeição de cadastro de plano sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC36 — Cadastro de plano sem Authorization

Método: POST – Endpoint: /account-plans

Entradas:

description = "Transporte"

Resultado Esperado:

- HTTP 401 Unauthorized

CT37 — Validar rejeição de plano com descrição duplicada

Técnica Utilizada:

- Transição de Estado

Justificativa:

Sistema muda do estado "plano inexistente" para "plano já cadastrado"; nova tentativa deve ser bloqueada.

TC37 — Cadastro de plano com descrição já existente

Método: POST — Endpoint: /account-plans

Pré-condição:

Plano "Alimentação" já cadastrado.

Entradas:

Authorization: Bearer token-ct-finansme

description = "Alimentação"

Resultado Esperado:

- HTTP 409 Conflict
- message = "Plano de conta já cadastrado."

CT38 — Validar listagem de planos de conta do usuário

Técnica Utilizada:

- Teste de Contrato

Justificativa:

Lista de planos deve conter elementos com chaves id_account_plan e description.

TC38 — Listar planos do usuário autenticado

Método: GET — Endpoint: /account-plans

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data não vazio
- cada item contém id_account_plan e description

CT39 — Validar rejeição de listagem de planos sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC39 — Listagem de planos sem Authorization

Método: GET — Endpoint: /account-plans

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized

Módulo Transactor — Cadastro e listagem

CT40 — Validar cadastro de transator com dados válidos

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Nome preenchido + token válido pertencem à classe válida.

TC40 — Cadastrar transator "Supermercado X"

Método: POST — Endpoint: /transactors

Entradas:

Authorization: Bearer token-ct-finansme

name = "Supermercado X"

Resultado Esperado:

- HTTP 201 Created
- message = "Transator cadastrado com sucesso."

CT41 — Validar rejeição de transator sem nome

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Body sem o campo name pertence à classe inválida.

TC41 — Cadastro de transator com body {}

Método: POST — Endpoint: /transactors

Entradas:

Authorization: Bearer token-ct-finansme

body = {}

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "name"

CT42 — Validar rejeição de cadastro de transator sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC42 — Cadastro de transator sem Authorization

Método: POST — Endpoint: /transactors

Entradas:

name = "Farmácia Y"

Resultado Esperado:

- HTTP 401 Unauthorized

CT43 — Validar rejeição de transator com nome duplicado

Técnica Utilizada:

- Transição de Estado

Justificativa:

Sistema transita de "transator livre" para "transator já cadastrado".

TC43 — Cadastro de transator já existente

Método: POST — Endpoint: /transactors

Pré-condição:

Transator "Supermercado X" já cadastrado.

Entradas:

Authorization: Bearer token-ct-finansme
name = "Supermercado X"

Resultado Esperado:

- HTTP 409 Conflict
- message = "Transator já cadastrado."

CT44 — Validar listagem de transatores do usuário

Técnica Utilizada:

- Teste de Contrato

Justificativa:

Cada elemento da lista deve possuir as chaves id_transactor e name.

TC44 — Listar transatores cadastrados

Método: GET — Endpoint: /transactors

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data não vazio
- cada item contém id_transactor e name

CT45 — Validar lista vazia quando não há transatores

Técnica Utilizada:

- Valor Limite

Justificativa:

Cenário limite (nenhum registro) deve retornar lista vazia, não erro.

TC45 — Listar transatores quando não há nenhum

Método: GET — Endpoint: /transactors

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data = []
- message = "Nenhum transator encontrado."

CT46 — Validar rejeição de listagem de transatores sem autenticação**Técnica Utilizada:**

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC46 — Listagem de transatores sem Authorization

Método: GET — Endpoint: /transactors

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized

Módulo Financial Movement — Criação**CT47 — Validar registro de movimentação válida (despesa)****Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

Body completo com type_movement_id=2 (despesa) e valor positivo é classe válida.

TC47 — Cadastrar despesa de 80,50

Método: POST — Endpoint: /financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

type_movement_id = 2

value = 80.50

movement_date = "2026-04-14"

reason = "Compra mensal"

account_id = 10

account_plan_id = 20

transator_id = 30

payment_method_id = 1

situation_id = 1

Resultado Esperado:

- HTTP 201 Created
- message = "Movimentação cadastrada com sucesso."

CT48 — Validar registro de movimentação válida (receita)

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Body completo com type_movement_id=1 (receita) é classe válida.

TC48 — Cadastrar receita de 3000,00 (Salário)

Método: POST – Endpoint: /financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

type_movement_id = 1

value = 3000.00

reason = "Salário"

Resultado Esperado:

- HTTP 201 Created
- type_movement_id recebido = 1

CT49 — Validar rejeição de lançamento com campo obrigatório ausente (reason)

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Body sem o campo reason pertence à classe inválida.

TC49 — Cadastro de movimentação sem reason

Método: POST – Endpoint: /financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

(body válido removendo "reason")

Resultado Esperado:

- HTTP 400 Bad Request
- message contém "reason"

CT50 — Validar rejeição de lançamento com valor negativo

Técnica Utilizada:

- Valor Limite

Justificativa:

Valor -100,00 está abaixo do limite inferior permitido (deve ser positivo).

TC50 — Cadastro de movimentação com value = -100,00

Método: POST — Endpoint: /financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

value = -100.00

Resultado Esperado:

- HTTP 400 Bad Request
- message = "O valor do lançamento deve ser positivo."

CT51 — Validar rejeição de lançamento sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC51 — Cadastro de movimentação sem Authorization

Método: POST — Endpoint: /financial-movements

Entradas:

(body válido sem token)

Resultado Esperado:

- HTTP 401 Unauthorized

CT52 — Validar rejeição de lançamento com tipo de movimentação inválido

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

type_movement_id = 99 não corresponde a nenhuma classe válida (1=receita, 2=despesa).

TC52 — Cadastro com type_movement_id = 99

Método: POST — Endpoint: /financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

type_movement_id = 99

Resultado Esperado:

- HTTP 400 Bad Request
- message = "Tipo de movimentação inválido."

Módulo Financial Movement — Listagem

CT53 — Validar listagem de movimentações do usuário autenticado

Técnica Utilizada:

- Particionamento de Equivalência
- Teste de Contrato

Justificativa:

Token válido + endpoint correto retornam o contrato esperado de movimentações.

TC53 — Listar movimentações com token válido

Método: GET — Endpoint: /user-financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data não vazio
- cada item contém id_financial_movement, type_movement_id, movement_date, due_date, payment_date, doc_num, transator_id, value, situation_id, account_id, account_plan_id e reason

CT54 — Validar lista vazia quando não há movimentações**Técnica Utilizada:**

- Valor Limite

Justificativa:

Cenário limite (nenhuma movimentação) deve retornar lista vazia.

TC54 — Listar movimentações de usuário sem registros

Método: GET — Endpoint: /user-financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data = []
- message = "Nenhuma movimentação encontrada."

CT55 — Validar listagem de múltiplas movimentações**Técnica Utilizada:**

- Teste Baseado em Cenário

Justificativa:

Cenário com várias movimentações; lista deve refletir todas.

TC55 — Listar 3 movimentações do usuário

Método: GET — Endpoint: /user-financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- data.length = 3

CT56 — Validar tipos dos campos de cada movimentação

Técnica Utilizada:

- Teste de Contrato

Justificativa:

Cada campo da movimentação deve respeitar o tipo declarado no contrato.

TC56 — Verificar tipos dos campos da movimentação

Método: GET — Endpoint: /user-financial-movements

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- id_financial_movement é int
- type_movement_id é int
- value é num
- reason é String

CT57 — Validar rejeição de listagem de movimentações sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC57 — Listagem de movimentações sem Authorization

Método: GET — Endpoint: /user-financial-movements

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized

Módulo Financial Movement — Atualização

CT58 — Validar alteração de lançamento existente com sucesso

Técnica Utilizada:

- Transição de Estado

Justificativa:

Estado do lançamento deve mudar para os novos valores após PUT bem-sucedido.

TC58 — PUT em /financial-movements/501

Método: PUT — Endpoint: /financial-movements/501

Pré-condição:

Movimentação 501 existente.

Entradas:

Authorization: Bearer token-ct-finansme

body completo com reason = "Compra mensal ajustada"

Resultado Esperado:

- HTTP 200 OK
- message = "Movimentação atualizada com sucesso."

CT59 — Validar rejeição de atualização sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC59 — PUT sem header Authorization

Método: PUT – Endpoint: /financial-movements/501

Entradas:

(body válido sem token)

Resultado Esperado:

- HTTP 401 Unauthorized

CT60 — Validar rejeição de atualização de lançamento inexistente

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

ID 9999 não está na partição de lançamentos existentes.

TC60 — PUT em /financial-movements/9999

Método: PUT – Endpoint: /financial-movements/9999

Entradas:

Authorization: Bearer token-ct-finansme

body válido

Resultado Esperado:

- HTTP 404 Not Found
- message = "Movimentação não encontrada."

CT61 — Validar rejeição de atualização com dados inválidos

Técnica Utilizada:

- Valor Limite

Justificativa:

value = -999,00 está abaixo do limite inferior permitido.

TC61 — PUT com value = -999,00

Método: PUT – Endpoint: /financial-movements/501

Entradas:

Authorization: Bearer token-ct-finansme
body = { value: -999.00 }

Resultado Esperado:

- HTTP 400 Bad Request
- message = "Dados inválidos para atualização."

CT62 — Validar atualização parcial — apenas o motivo do lançamento**Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

Atualização parcial é variação válida do contrato.

TC62 — PUT enviando apenas o campo reason

Método: PUT — Endpoint: /financial-movements/501

Entradas:

Authorization: Bearer token-ct-finansme
body = { reason: "Ajuste de compra" }

Resultado Esperado:

- HTTP 200 OK
- message = "Movimentação atualizada com sucesso."

Módulo Financial Movement — Exclusão**CT63 — Validar exclusão de lançamento existente****Técnica Utilizada:**

- Transição de Estado

Justificativa:

Sistema transita de "lançamento existente" para "lançamento removido" após DELETE bem-sucedido.

TC63 — DELETE em /financial-movements/501

Método: DELETE — Endpoint: /financial-movements/501

Pré-condição:

Movimentação 501 existente.

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- message = "Movimentação excluída com sucesso."

CT64 — Validar rejeição de exclusão sem autenticação

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC64 — DELETE sem Authorization

Método: DELETE — Endpoint: /financial-movements/501

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized

CT65 — Validar rejeição de exclusão de lançamento inexistente**Técnica Utilizada:**

- Particionamento de Equivalência

Justificativa:

ID 9999 não está na partição de lançamentos existentes.

TC65 — DELETE em /financial-movements/9999

Método: DELETE — Endpoint: /financial-movements/9999

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 404 Not Found

- message = "Movimentação não encontrada."

CT66 — Validar rejeição de exclusão com token inválido (sem permissão)**Técnica Utilizada:**

- Tabela de Decisão

Justificativa:

Token inválido autenticado em outro contexto não deve ter permissão de exclusão.

TC66 — DELETE com token de outro usuário

Método: DELETE — Endpoint: /financial-movements/501

Entradas:

Authorization: Bearer token-invalido

Resultado Esperado:

- HTTP 403 Forbidden

- message = "Acesso negado. Você não tem permissão para excluir este lançamento."

Módulo AI Advisor — Conselheiro de IA**CT67 — Validar geração de resposta do conselheiro de IA para usuário autenticado**

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Token válido + endpoint correto pertencem à classe válida; espera-se resposta textual.

TC67 — GET /ai-response com token válido

Método: GET — Endpoint: /ai-response

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- HTTP 200 OK
- AI_response é String

CT68 — Validar rejeição de consulta ao conselheiro sem autenticação**Técnica Utilizada:**

- Tabela de Decisão

Justificativa:

Recurso protegido sem token deve retornar 401.

TC68 — GET /ai-response sem Authorization

Método: GET — Endpoint: /ai-response

Entradas:

(sem Authorization)

Resultado Esperado:

- HTTP 401 Unauthorized

CT69 — Validar que a resposta da IA é uma string não vazia**Técnica Utilizada:**

- Valor Limite
- Teste de Contrato

Justificativa:

AI_response = "" representaria limite inferior inválido; deve ter ao menos 1 caractere.

TC69 — Verificação de não-vazio da AI_response

Método: GET — Endpoint: /ai-response

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- AI_response é String
- AI_response.isNotEmpty

CT70 — Validar chave AI_response no payload de resposta

Técnica Utilizada:

- Teste de Contrato

Justificativa:

O contrato exige que a resposta contenha exatamente a chave `AI_response`.

TC70 — Verificação da chave `AI_response`

Método: GET — Endpoint: `/ai-response`

Entradas:

Authorization: Bearer token-ct-finansme

Resultado Esperado:

- `response.keys` contém "`AI_response`"

CT71 — Validar rejeição de consulta ao conselheiro com token expirado**Técnica Utilizada:**

- Transição de Estado

Justificativa:

Token muda de "válido" para "expirado"; sistema deve recusar.

TC71 — GET `/ai-response` com token expirado

Método: GET — Endpoint: `/ai-response`

Entradas:

Authorization: Bearer token-expirado

Resultado Esperado:

- HTTP 401 Unauthorized
- message contém "Token"

3. Tabela Consolidada de Técnicas

Condição	Técnica(s)
CT01	Particionamento de Equivalência
CT02	Transição de Estado
CT03	Particionamento de Equivalência
CT04	Particionamento de Equivalência
CT05	Valor Limite
CT06	Valor Limite
CT07	Particionamento de Equivalência
CT08	Particionamento de Equivalência
CT09	Particionamento de Equivalência
CT10	Particionamento de Equivalência + Valor Limite
CT11	Particionamento de Equivalência
CT12	Teste de Contrato
CT13	Particionamento de Equivalência
CT14	Tabela de Decisão
CT15	Transição de Estado
CT16	Particionamento de Equivalência + Valor Limite
CT17	Particionamento de Equivalência
CT18	Particionamento de Equivalência
CT19	Valor Limite
CT20	Particionamento de Equivalência
CT21	Valor Limite
CT22	Tabela de Decisão
CT23	Particionamento de Equivalência
CT24	Particionamento de Equivalência + Teste de Contrato
CT25	Teste Baseado em Cenário
CT26	Valor Limite
CT27	Tabela de Decisão
CT28	Teste de Contrato
CT29	Particionamento de Equivalência + Teste de Contrato
CT30	Transição de Estado + Teste Baseado em Cenário
CT31	Transição de Estado + Teste Baseado em Cenário
CT32	Teste Baseado em Cenário + Transição de Estado
CT33	Tabela de Decisão
CT34	Particionamento de Equivalência
CT35	Particionamento de Equivalência
CT36	Tabela de Decisão
CT37	Transição de Estado

Condição	Técnica(s)
CT38	Teste de Contrato
CT39	Tabela de Decisão
CT40	Particionamento de Equivalência
CT41	Particionamento de Equivalência
CT42	Tabela de Decisão
CT43	Transição de Estado
CT44	Teste de Contrato
CT45	Valor Limite
CT46	Tabela de Decisão
CT47	Particionamento de Equivalência
CT48	Particionamento de Equivalência
CT49	Particionamento de Equivalência
CT50	Valor Limite
CT51	Tabela de Decisão
CT52	Particionamento de Equivalência
CT53	Particionamento de Equivalência + Teste de Contrato
CT54	Valor Limite
CT55	Teste Baseado em Cenário
CT56	Teste de Contrato
CT57	Tabela de Decisão
CT58	Transição de Estado
CT59	Tabela de Decisão
CT60	Particionamento de Equivalência
CT61	Valor Limite
CT62	Particionamento de Equivalência
CT63	Transição de Estado
CT64	Tabela de Decisão
CT65	Particionamento de Equivalência
CT66	Tabela de Decisão
CT67	Particionamento de Equivalência
CT68	Tabela de Decisão
CT69	Valor Limite + Teste de Contrato
CT70	Teste de Contrato
CT71	Transição de Estado

4. Tabela Consolidada de Casos de Teste

ID	Caso
TC01	Cadastro de usuário com dados válidos
TC02	Cadastro com e-mail já existente
TC03	Cadastro sem nome no body
TC04	Cadastro com formato de e-mail inválido
TC05	Cadastro com senha menor que 8 caracteres
TC06	Cadastro com body vazio
TC07	Login com credenciais corretas
TC08	Login com senha incorreta
TC09	Login com e-mail não cadastrado
TC10	Login com e-mail e senha em branco
TC11	Login sem o campo email
TC12	Verificação das chaves de resposta do login
TC13	Atualização de dados com token válido
TC14	Atualização sem header Authorization
TC15	Atualização com token expirado
TC16	Atualização com campos vazios
TC17	Atualização enviando somente o campo name
TC18	Cadastrar conta "Carteira" com saldo 150,00
TC19	Cadastrar conta "Poupança" com saldo 0,00
TC20	Cadastro de conta sem o campo description
TC21	Cadastro de conta com saldo negativo
TC22	Cadastro de conta sem header Authorization
TC23	Cadastro de conta com Bearer Token correto
TC24	Listar contas com token válido
TC25	Listar 3 contas do usuário
TC26	Listar contas de usuário sem contas
TC27	Listar contas sem header Authorization
TC28	Verificar tipos dos campos da conta listada
TC29	Consultar saldo com token válido
TC30	GET saldo → POST receita → GET saldo
TC31	GET saldo → POST despesa → GET saldo
TC32	Sequência saldo → receita → saldo → despesa → saldo
TC33	Consulta de saldo sem Authorization
TC34	Cadastrar plano "Alimentação"
TC35	Cadastro de plano com body {}
TC36	Cadastro de plano sem Authorization
TC37	Cadastro de plano com descrição já existente

ID	Caso
TC38	Listar planos do usuário autenticado
TC39	Listagem de planos sem Authorization
TC40	Cadastrar transator "Supermercado X"
TC41	Cadastro de transator com body {}
TC42	Cadastro de transator sem Authorization
TC43	Cadastro de transator já existente
TC44	Listar transatores cadastrados
TC45	Listar transatores quando não há nenhum
TC46	Listagem de transatores sem Authorization
TC47	Cadastrar despesa de 80,50
TC48	Cadastrar receita de 3000,00 (Salário)
TC49	Cadastro de movimentação sem reason
TC50	Cadastro de movimentação com value = -100,00
TC51	Cadastro de movimentação sem Authorization
TC52	Cadastro com type_movement_id = 99
TC53	Listar movimentações com token válido
TC54	Listar movimentações de usuário sem registros
TC55	Listar 3 movimentações do usuário
TC56	Verificar tipos dos campos da movimentação
TC57	Listagem de movimentações sem Authorization
TC58	PUT em /financial-movements/501
TC59	PUT sem header Authorization
TC60	PUT em /financial-movements/9999
TC61	PUT com value = -999,00
TC62	PUT enviando apenas o campo reason
TC63	DELETE em /financial-movements/501
TC64	DELETE sem Authorization
TC65	DELETE em /financial-movements/9999
TC66	DELETE com token de outro usuário
TC67	GET /ai-response com token válido
TC68	GET /ai-response sem Authorization
TC69	Verificação de não-vazio da AI_response
TC70	Verificação da chave AI_response
TC71	GET /ai-response com token expirado

5. Conclusão da Etapa

As condições de teste identificadas no Documento A foram derivadas em casos de teste completos utilizando as técnicas formais definidas pela ISO/IEC/IEEE 29119-4, abrangendo Particionamento de Equivalência, Valor Limite, Transição de Estado, Teste Baseado em Cenário, Tabela de Decisão e Teste de Contrato.

Os 71 casos de teste produzidos cobrem os 6 módulos funcionais do aplicativo Finansme Flutter (User, Account, Account Plan, Transactor, Financial Movement e AI Advisor) e estão implementados como testes automatizados em Dart usando flutter_test e o servidor mock TestApiServer, prontos para execução contínua via comando flutter test.